

TUGAS

BASIS DATA LANJUT

Sistem Informasi Manajemen Perpustakaan Berbasis API

PERANCANGAN DAN IMPLEMENTASI

Skema Basis Data, Migrasi Laravel, *Eloquent Model*,
RESTful API (60 Endpoint), *Controller*, Pengujian, dan Keamanan Sistem

Nama Kelompok:

1. Diana Indah Sayekti (1412240029)
2. Moh. Fatih Nurrudin (1412240052)
3. Shofin Zahrotus T. (1412240080)
4. Siti Mu'allimah (1412240082)
5. Nurlaili Islah Z. (1412240099)

Studi Kasus : Sistem Perpustakaan Digital

Kelas : TIF 2024 A

Dosen : Andy Haryoko, S.T., M.T.

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS PGRI RONGGOLawe TUBAN
TAHUN AKADEMIK 2024/2025

LAPORAN SINGKAT

Latar Belakang, Rancangan Solusi, Kendala Teknis, dan Pengujian

A. Latar Belakang dan Kebutuhan Fitur

Sistem Informasi Manajemen Perpustakaan ini dikembangkan untuk menjawab permasalahan pada tata kelola perpustakaan konvensional yang umumnya masih bersifat manual dan belum terintegrasi. Kebutuhan fungsional utama yang mendasari pengembangan sistem ini meliputi:

1. **Pencatatan Peminjaman dan Pengembalian:** Otomatisasi pencatatan peminjaman serta pengembalian buku yang didukung oleh mekanisme transaksi basis data guna menjaga konsistensi stok barang.
2. **Manajemen Denda Terstruktur:** Sistem pengelolaan denda yang dipisahkan ke dalam tiga entitas (tabel *dendas*, *pembayaran_dendas*, dan *riwayat_status_dendas*) guna memfasilitasi pembayaran secara bertahap (parsial) serta rekam jejak (*audit log*).
3. **Reservasi Buku:** Fitur pemesanan untuk buku yang sedang dipinjam oleh anggota lain, dilengkapi dengan aturan batas waktu klaim.
4. **Otorisasi Berbasis Peran:** Pembagian hak akses ke dalam tiga level (admin, petugas, dan anggota) menggunakan *middleware CheckRole* guna mengamankan setiap *endpoint* yang tersedia.
5. **Pengendalian Konkurensi (*Concurrency Control*):** Penanganan kondisi balapan (*race condition*) pada stok buku ketika terjadi peminjaman secara bersamaan melalui `DB::transaction()` dan mekanisme *atomic decrement*.

B. Rancangan Solusi

Sistem ini dirancang sebagai sebuah *RESTful API* berarsitektur monolitik menggunakan kerangka kerja Laravel 11.x, dengan rincian arsitektur sebagai berikut:

1. **Skema Basis Data:** Terdiri dari 14 tabel (termasuk *users*) yang telah memenuhi kaidah normalisasi 3NF. Skema ini mencakup 10 tabel eksisting (*kategoris*, *penerbits*, *raks*, *anggotas*, *petugas*, *bukus*, *peminjamans*,

detail_peminjamans, pengembalians, reservasis) dan 3 tabel baru terkait denda (dendas, pembayaran_dendas, riwayat_status_dendas).

2. **RESTful API:** Menyediakan 62 *endpoint* dengan format respons JSON yang terstandarisasi (success, message, data, errors). Autentikasi dikelola menggunakan *Bearer Token* dari Laravel Sanctum, sementara otorisasi ditangani oleh *middleware* role.
3. **Eloquent Model:** Mengimplementasikan 13 model yang saling terhubung melalui relasi seperti *belongsTo* dan *hasMany*. Selain itu, digunakan *query scope* untuk efisiensi penyaringan data (contoh: *scopeAktif*, *scopeDipinjam*, *scopeDendaPending*).
4. **Transaksi Basis Data (Database Transaction):** Operasi yang melibatkan manipulasi multitabel (seperti peminjaman, pengembalian, dan denda) dibungkus di dalam `DB::transaction()` untuk menjamin prinsip atomisitas.
5. **Eager Loading:** Metode `with()` diterapkan secara konsisten pada seluruh *endpoint* `index()` dan `show()` untuk mengoptimalkan performa dan mencegah isu *N+1 query*.

C. Kendala Teknis dan Pemecahannya

Selama siklus pengembangan, terdapat beberapa kendala teknis yang ditemukan dan berhasil diatasi:

1. **Race Condition pada Stok Buku:** Munculnya kemungkinan dua permintaan peminjaman yang masuk secara bersamaan sehingga melebihi kapasitas stok aktual. **Solusi:** Implementasi `DB::transaction()` yang dipadukan dengan metode `lockForUpdate()` serta validasi stok ketat sebelum transaksi diselesaikan.
2. **Pembayaran Denda Parsial:** Terdapat skenario di mana denda tidak dibayar lunas dalam satu kali transaksi. **Solusi:** Pemisahan entitas pembayaran ke dalam tabel `pembayaran_dendas` dan penambahan kolom `sisa_denda` pada tabel utama denda untuk melacak sisa tagihan.
3. **Duplikasi Buku dalam Satu Nota Peminjaman:** Sistem berpotensi mencatat buku yang sama secara berulang dalam satu resi peminjaman. **Solusi:** Penambahan *constraint* `UNIQUE` pada kombinasi kolom (`id_peminjaman`, `id_buku`) di tingkat basis data.

4. **Tumpang Tindih (*Overlapping*) Reservasi:** Buku yang telah direservasi tidak boleh dipinjamkan kepada anggota lain. **Solusi:** Penambahan logika validasi status reservasi pada saat transaksi peminjaman berlangsung.

D. Skenario Pengujian

Pengujian fungsional sistem dilakukan menggunakan PHPUnit 11. Skenario *Feature Test* ini difokuskan pada tiga alur bisnis utama:

1. Alur Peminjaman Buku:

- **Skenario Berhasil:** *Request* POST /api/peminjaman dengan *payload* valid menghasilkan status 201 Created, stok buku berkurang, dan detail transaksi tercatat.
- **Skenario Gagal:** *Request* tanpa token (401 Unauthorized), stok buku habis (409 Conflict), atau anggota berstatus nonaktif (422 Unprocessable Entity).
- **Validasi Integritas:** Upaya duplikasi kode_peminjaman yang sama akan ditolak dengan status 409 Conflict.

2. Alur Pengembalian Buku:

- **Skenario Berhasil:** *Request* POST /api/pengembalian dengan data valid menghasilkan status 201 Created dan mengembalikan jumlah stok buku.
- **Skenario Gagal:** Detail pengembalian tidak sesuai (422), atau kondisi buku dilaporkan rusak/hilang yang memicu pencatatan denda otomatis.
- **Validasi Duplikasi:** Percobaan pengembalian ulang untuk detail transaksi yang sudah berstatus kembali akan ditolak (422).

3. Alur Pembayaran Denda:

- **Skenario Berhasil:** *Request* POST /api/denda/{id}/pembayaran dengan nominal jumlah_bayar valid menghasilkan status 201 Created dan memperbarui informasi sisa denda.
- **Skenario Gagal:** Nominal pembayaran melebihi sisa denda (422) atau upaya pembayaran pada denda yang sudah berstatus lunas (422).

Seluruh rangkaian pengujian dieksekusi melalui perintah `php artisan test`. Hasilnya menunjukkan bahwa 12 metode pengujian telah berjalan dengan sukses dan seluruhnya berstatus *passed*.